Foundation Time Series Models: An Application for Retail Forecasting

Mariana Menchero

mariana@nixtla.io

F4SG Learning Labs 10

About me

Senior forecaster at Nixtla

Developer of the nixtlar package

Previously, senior forecaster at Ritter Dragon, a consulting company in Mexico City

MIT Micromasters in Supply Chain Management credential holder

Forecasting enthusiast since 2019 ♥

Program for today

Part 1: Foundation Models for Time Series Forecasting

1.1 Overview of foundation models

1.2 Introduction to TimeGPT

1.3 The nixtlar package

Part 2: Hands-On Example with a Retail Dataset

2.1 Forecasting using TimeGPT via nixtlar

2.2 Best practices for retail forecasting

All the code and slides used in this workshop can be found in this repository:

https://github.com/MMenchero/nixtlar_f4sg

To access TimeGPT for free, sign up here:

https://dashboard.nixtla.io/sign_in

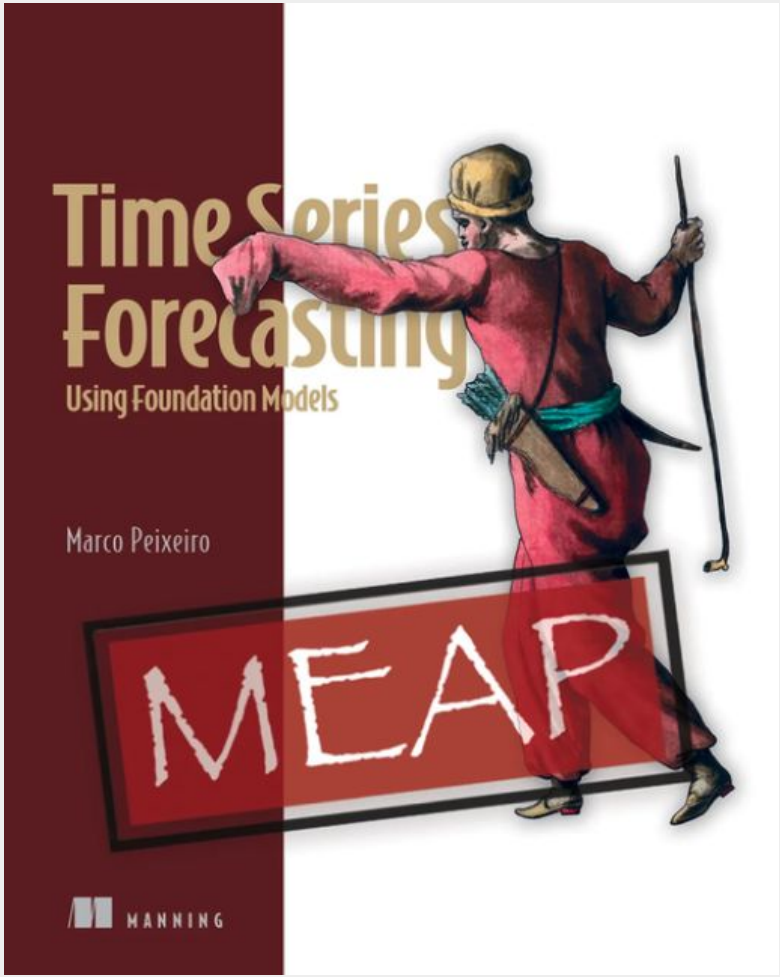
For inquiries:

support@nixtla.io

Part 1: Foundation Models For Time Series Forecasting

1.1 Overview of Foundation Models

Main reference for this section



What are Foundation Models?

In the context of time series forecasting, a foundation model is a machine learning model that has been trained on a very large dataset and that is capable of predicting series not seen during training.

Some foundations models are also capable of perform other time series related tasks, such as anomaly detection or classification.

Types of Foundation Models

There are two main **categories**:

1. Foundation models that are specifically designed to handle time series data.
   - TimeGPT
   - Lag-LLaMa
   - Chronos
   - TimesFM
   - Moirai

2. LLM that have been extended to handle time series problems.
   - TimeLLM
   - PromptCast

Why Do Foundation Models work?

Foundation models are trained on very large amounts of data. These models often weight many gigabytes and are trained for days.

A single foundation model can predict series with different properties, such as frequencies, trends, or seasonal periods. The idea is that even though these series have never seen your data, they have seen other series that are “similar enough”.

Fine-tuning vs Zero-shot

Foundation models can also be **fine-tuned**, meaning that certain parts of the model are trained on your specific data. In this process, only a subset of the model's parameters is updated, rather than the entire model, since full retraining would take very long.

The **zero-shot** version of a foundation model refers to its original state, without any fine-tuning.

Beware of overfitting!

How Do Foundation Models Work?

Most foundation models, including TimeGPT, are built on the **transformer architecture**.

1. **Embedding Layer**: Converts the time series into a format the model can understand.
2. **Positional Encoding**: Adds temporal information so that the order of the time steps is recognized.
3. **Encoder**: Uses self-attention to learn patterns and dependencies across time steps.
4. **Decoder**: Predicts step-by-step, feeding each output back into the model.

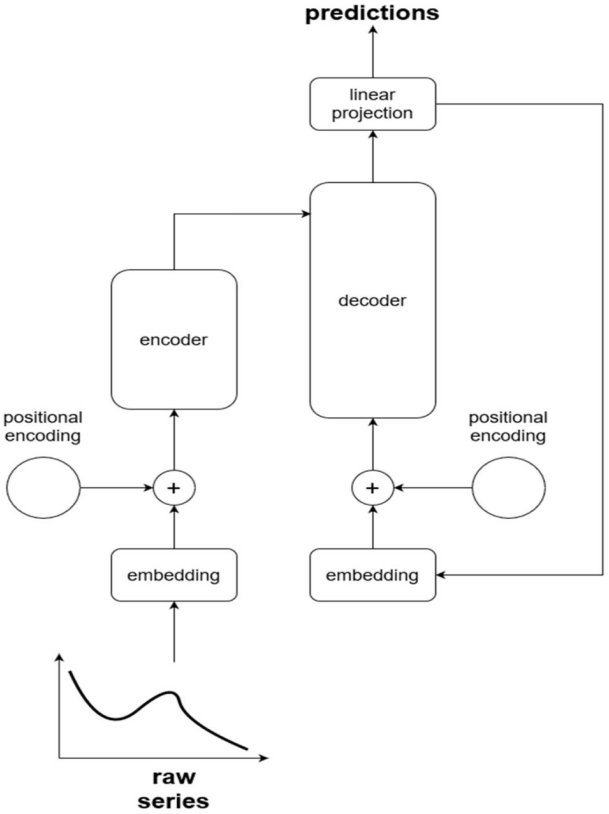


Image taken from Peixeiro (2025)

1. Feeding the encoder

- The **raw time series** data is first processed before being sent into the transformer.
- The **embedding layer** converts the data into **tokens** (which can be individual values or small windows of values).
- The model **learns an embedding**, which is an abstract, numerical representation of the time series. This embedding captures essential patterns and features.

2. Positional Encoding

- Since time series data depends on the order of observations, the transformer must know the position of each token.
- **Positional encoding** is added to the embeddings so the model understands **time order**.
- It uses **sinusoidal functions (sine and cosine waves)** at different frequencies to encode position, similar to how clock hands represent time at different granularities (hour, minute, second, etc.).

3. Encoder

The **encoder** is a stack of layers, each containing:

- A **self-attention mechanism** (helps identify relationships between different time points).
- A **feed-forward neural network** (helps process and refine the representation).

Self-attention helps the model learn **dependencies** within the time series, such as trends and seasonality.

Multi-headed attention allows the model to focus on different patterns in the data:

  - One head might detect **trends**.
  - Another head might focus on **seasonality**.

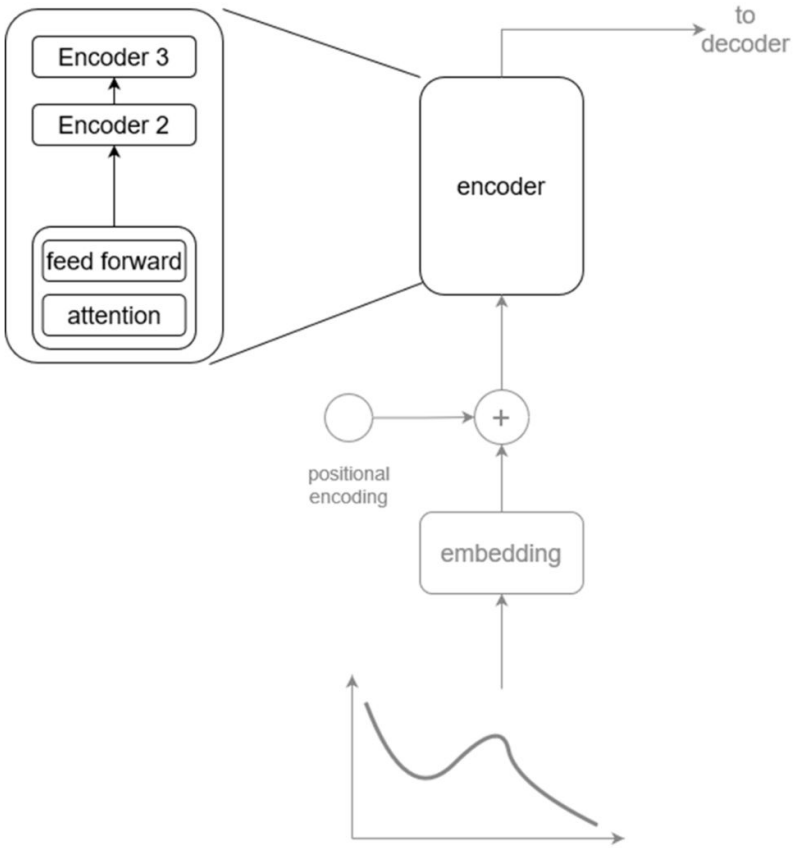


Image taken from Peixeiro (2025)

4. Decoder

The **decoder** is similar to the encoder but has additional components:

- A **masked multi-headed attention layer**, which prevents the model from looking at future data.
- Another **attention layer**, which connects the decoder to the encoder's output.
- A **feed-forward network** that refines the learned information.
- **Normalization layers** that keep the training stable.

The **decoder generates predictions** by using the learned embeddings and feeding them through this architecture.

After passing through the decoder, predictions are made using a **linear projection layer**.

The first prediction is used to inform the next one, repeating the process until the entire forecast horizon is covered.

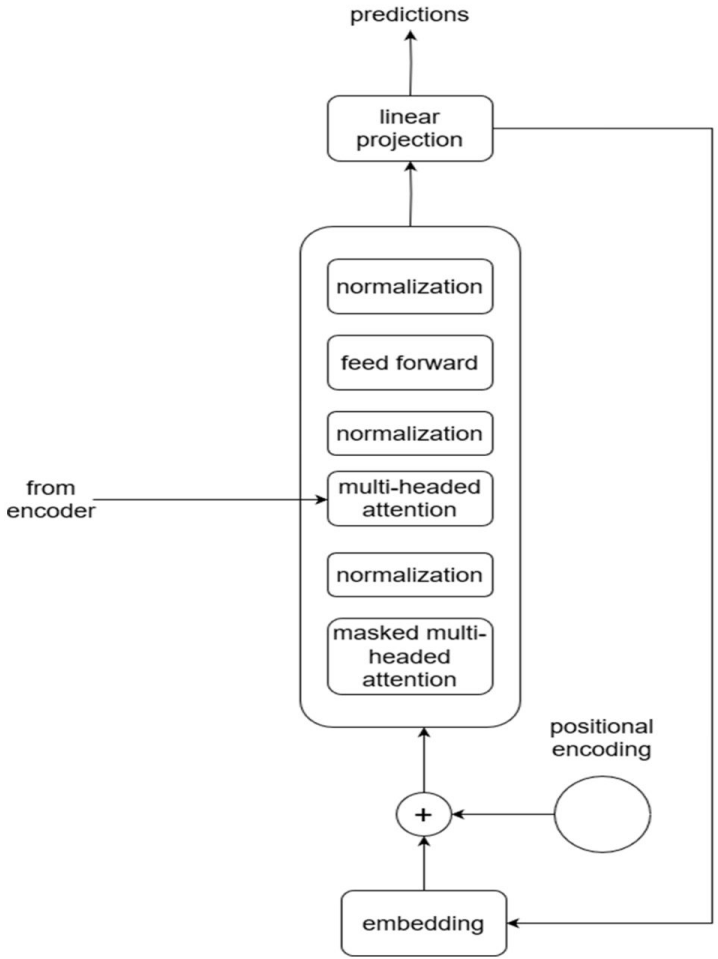


Image taken from Peixeiro (2025)

Summary

- The **encoder** learns **patterns** and **relationships** in the time series.
- The **decoder** generates predictions based on what the encoder has learned.
- **Embedding layers** transform the data into a format that the model can understand.
- **Positional encoding** ensures the model understands the sequence order.
- Some foundation models use **only the encoder, only the decoder, or a modified version** of the transformer.

Advantages	Disadvantages
Requires minimal code and forecasting expertise.	Your specific case might not be well-represented.
Can generate forecasts even with few observations.	No direct access to model internals for customization and black-box model.
Applicable to many forecasting scenarios.	If accessed via API, service outages can disrupt availability. Check http://status.nixtla.io/

1.2 Introduction to TimeGPT

The first foundation model for time series forecasting.

Released by Azul Garza, Cristian Challu and Max Mergenthaler-Canseco on May 2024 (research paper)

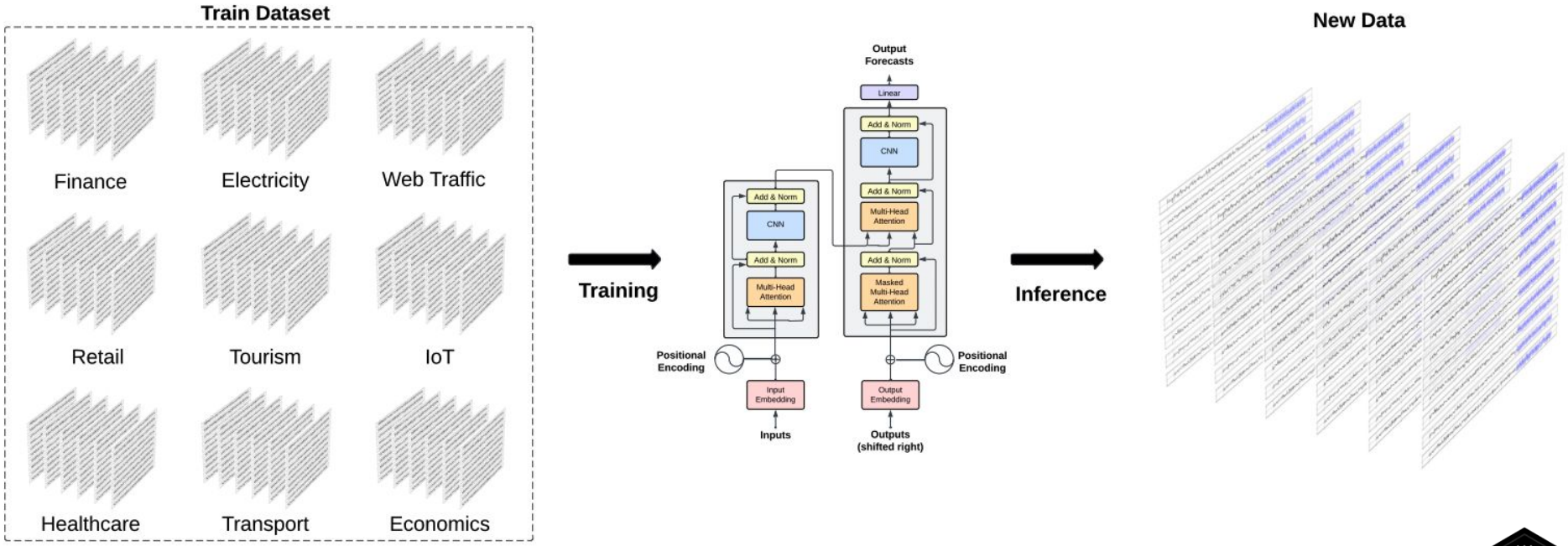
TimeGPT model

TimeGPT is a **generative pre-trained transformer**, built on the transformer architecture described in the previous section.

It is **generative** because it is trained to take an input sequence and generate an output sequence in an **autoregressive** manner, so that each predicted element is fed back into the model to generate the next step.

Additionally, it has been **pre-trained** on a very large dataset - *more on this later*

TimeGPT model



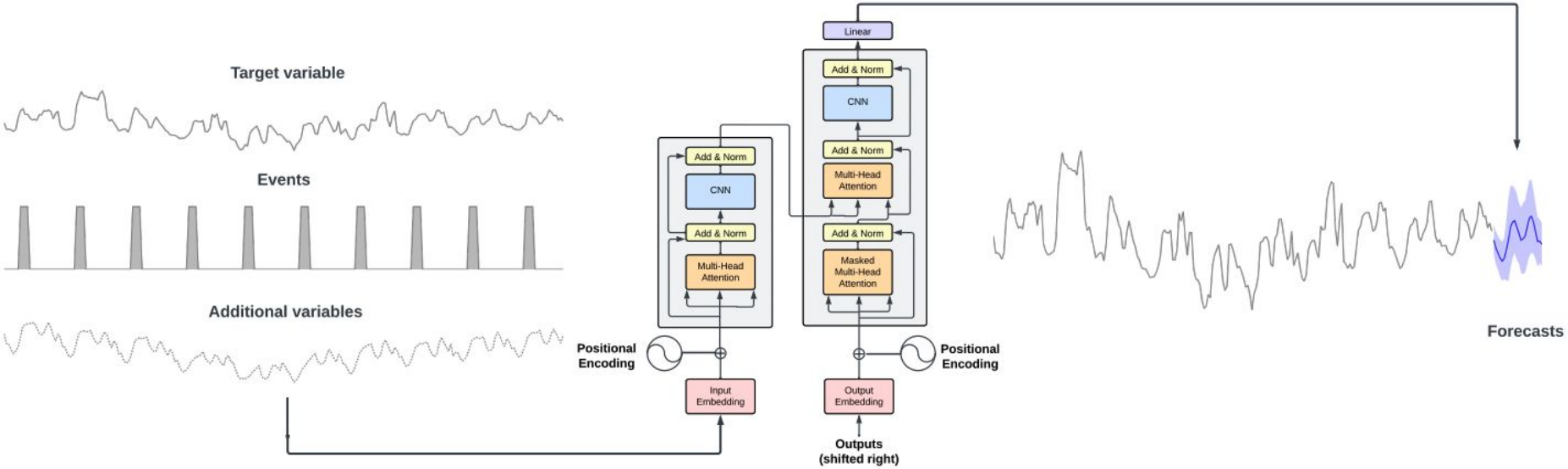
TimeGPT (formal) objective

Estimate the following conditional distribution:

$$\mathbb{P}\left(y_{[t+1:t+h]} \middle| y_{[0:t]}, x_{[0:t+h]}\right) = f_\theta\left(y_{[0:t]}, x_{[0:t+h]}\right)$$

$$\begin{aligned}
h &= \text{the forecast horizon} \\
y_{[0:t]} &= \text{the current and past values of the series} \\
y_{[t+1:t+h]} &= \text{the future values of the series over the horizon} \\
x_{[0:t+h]} &= \text{the exogenous features} \\
f_\theta &= \text{the future that TimeGPT is trying to estimate from the training data} \\
\theta &= \text{the parameters of } f
\end{aligned}$$

TimeGPT (formal) objective



TimeGPT Training Data

- **Large-scale dataset**: The training set contains over 100 billion data points.
- **Diverse domains**: Time series come from multiple industries and applications.
- **Varied series**: The dataset includes time series with different scales, lengths, and temporal patterns (e.g., multiple seasonalities and various types of trends).
- **Minimal preprocessing**: Only missing values were interpolated using a proprietary method. Hence, it has seen data with noise and outliers.
- **Computationally intensive training** – Training took several days on a cluster of NVIDIA A10G GPUs.

TimeGPT Capabilities

Supports the following features:

- Multiple series concurrently
- Fine-tuning
- Exogenous variables (historic and future)
- Cross-validation
- Anomaly detection
- Prediction intervals

All of these features are available through *nixtla*, the Python SDK.

1.3 The nixtlar Package

Available on CRAN and on GitHub (development version)

THE NIXTLAVERSE

The R ecosystem for time series forecasting

Part 2: Hands-On Example with a Retail Dataset

VN1 Forecasting Competition

September 2024 - October 2024

Weekly retail dataset with 15,053 series and a forecast horizon of 13 weeks

Organized by Nicolas Vandeput on the datasource.ia platform and sponsored by Flieber, SupChains and Syrup

Evaluation metric

$$score = \frac{\sum_{i=1}^{N} |\hat{y} - y|}{\sum_{i=1}^{N} \hat{y} - y}$$

What’s Next

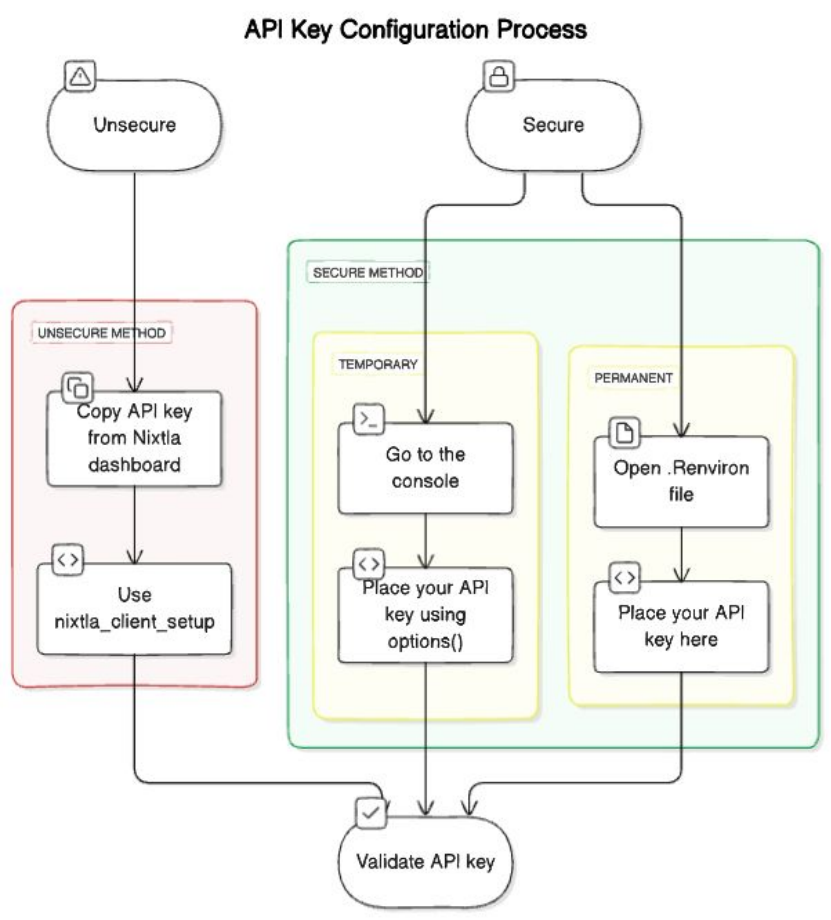
In this section, I will explain:

- How to set up your API key
- Data requirements and required pre-processing steps
- How to use TimeGPT
- How to leverage TimeGPT’s features in nixtlar

Along the way, I’ll share stories from my experience as a retail forecaster to give you real-world insights and practical tips - *do I have some stores to tell!*

Set up your API key

Get yours at:
https://dashboard.nixtla.io/sign_in



Storytime

Exogenous variables

1. Pet shops: *Prices are not real prices*
2. Clothing stores: *Surprise promotions*

Fine-tuning

- **finetune_steps**: The number of training iterations applied to the input data minimizing the forecasting error.
- **finetune_depth**: A integer value from 1 to 5, indicating the set of parameters that will be fine-tuned. When set to 1, only a small set of parameters is adjusted, and when set to 5 all of them are.
- **finetune_loss**: The loss function used during fine-tuning.
  - "default" - a proprietary loss function that is robust to outliers
  - "mae" - mean absolute error
  - "mse" - mean squared error
  - "rmse" - root mean squared error
  - "mape" - mean absolute percentage error
  - "smape" - symmetric mean absolute percentage error

Today’s Takeaways

Takeaways

Foundation models are a new tool for time series forecasting. They may or may not be the best fit for all your series, but they're worth experimenting with.

Forecasting is not an end in itself, decision-making is. Don't focus just on KPIs and think about how your forecasts impact the decisions you make.

Retail forecasting comes with challenges like handling a large number of series, dealing with intermittency, and working within tight time constraints. It also requires a good mix of technical skills and business knowledge.

TimeGPT offers strong performance compared to other forecasting methods and is relatively easy to implement.

Get started, experiment, and keep iterating!

nixtla

Takeaways

In this workshop, you learned how to use nixtlar to:

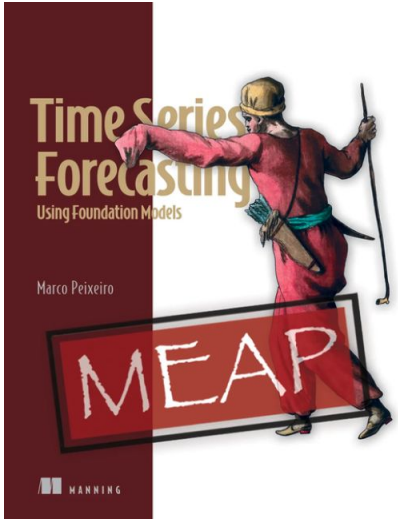
- ✔ Visualize your data – *always the first step*
- ✔ Implement TimeGPT (setting up your API key & data requirements)
- ✔ Build long-horizon forecasts with TimeGPT
- ✔ Fine-tune your models with several parameters or via the AutoTimeGPT
- ✔ Incorporate exogenous variables (historic and future)
- ✔ Generate probabilistic forecasts to estimate uncertainty
- ✔ Detect anomalies

Resources

Official documentation

nixtlar: https://nixtla.github.io/nixtlar/ ⭐

nixtla: https://docs.nixtla.io/



On foundation models

Book: Peixeiro, Marco. *Time Series Forecasting Using Foundation Models*. Manning Publications, 2025 (upcoming, now in early access)

Podcast: “Panel on Foundation Models with Azul Garza Ramírez and Mononito Goswami.” *Forecasting Impact.*

Resources

On retail forecasting and supply chain

Lokad's blog: https://www.lokad.com/blog/

Book: Vandeput, Nicolas. *Demand Forecasting Best Practices*. Simon and Schuster, 2023.

VN1 Competition

Vandeput, Nicolas. "VN1 Forecasting - Accuracy Challenge." DataSource.ai, DataSource, 3 Oct. 2024, https://www.datasource.ai/en/home/data-science-competitions-for-startups/phase-2-vn1-forecasting-accuracy-challenge/description

Q&A

Stay in touch: https://www.linkedin.com/in/marianamencherogarcia/

Open an issue in nixtlar: https://github.com/Nixtla/nixtlar/issues